

P-RIO: A METODOLOGIA RIO SOBRE PVM

Enrique Vinicio Carrera E.

Dpto. Eng. Elétrica - PUC/Rio

Cx.Postal 38063 - CEP 22453-900, Rio de Janeiro-RJ

E-mail: vinicio@ele.puc-rio.br

Orlando Loques

CAA - P.G. em Computação Aplicada e Automação
e Dpto. de Eng. Elétrica - UFF

Rua Passo da Pátria, 156-CEP 24210-240, Niterói-RJ

Julius Leite

Resumo

O presente trabalho descreve o suporte da metodologia de construção de *software* embutida no ambiente RIO^[1], sobre a plataforma PVM^[2]. Esta implementação visa reunir num só ambiente as facilidades de alto nível para a construção de sistemas paralelos e distribuídos providas por RIO, à portabilidade oferecida pelas primitivas suportadas pela máquina virtual PVM, que é disponível em um grande número de arquiteturas de *hardware*. Esta estratégia permite que usuários utilizem abstrações poderosas de modularização, interfaceamento e configuração sem a necessidade de envolvimento com os detalhes de baixo nível normalmente associados ao uso direto da plataforma PVM. Além disso, a metodologia RIO facilita a reutilização do *software* e a manutenção de sistemas, permitindo maior eficiência no processo de desenvolvimento de aplicações.

Abstract

This paper describes the support of the software construction methodology embedded in a version of the RIO environment built on the PVM platform. This strategy attempts to combine in a single environment the high-level facilities provided by RIO to make parallel and distributed systems, and the portability offered by the PVM virtual machine that is supported in several hardware architectures. This allows users to take advantage of the powerful modularity, interfacing and configuration abstractions provided by RIO without needing to be involved with low-level details generally associated with the direct use of the PVM system. Besides this, the RIO methodology facilitates software reuse and system maintenance, improving the efficiency of the applications development process.

1. INTRODUÇÃO

PVM (Parallel Virtual Machine) é uma plataforma de *software* que simplifica a programação de um conjunto heterogêneo de computadores interligados em rede^[3], fornecendo ao programador a visão de uma máquina única. As máquinas individuais podem ser multiprocessadores de memória compartilhada ou memória local, supercomputadores vetoriais, estações gráficas especializadas, ou *workstations* escalares, e podem estar interconectadas através de uma variedade de redes, tais como Ethernet, Token Ring, FDDI, etc.

Os programas de usuário, escritos em C ou Fortran, podem acessar os recursos da plataforma através das bibliotecas de funções PVM para realizar atividades de baixo nível como iniciação de processos, transmissão e recepção de mensagens, sincronização mediante barreiras ou

encontros (*rendezvous*), etc. Os usuários podem opcionalmente controlar a localização da execução dos componentes de uma aplicação, assim como monitorar o estado de um *host* determinado. O sistema PVM manipula transparentemente o roteamento das mensagens, a conversão de dados entre arquiteturas incompatíveis, e outras tarefas que são necessárias para a operação num ambiente heterogêneo de rede^[4].

O pacote de *software* associado ao PVM é de livre distribuição e está convertendo-se num padrão na área de processamento paralelo. Ele está sendo desenvolvido, desde 1989, no Oak Ridge National Laboratory (ORNL), University of Tennessee (UT), Emory University e Carnegie Mellon University. O projeto é atualmente financiado pelo U.S. Department of Energy, pela National Science Foundation e pelo Estado de Tennessee^[5].

Por outro lado, o ambiente **RIO** (*Reconfigurable Interconnectable Objects*), em desenvolvimento por um grupo de pesquisadores e alunos de pós-graduação, no DEE-PUC/RJ e no CAA-UFF, oferece facilidades para a programação e a configuração de componentes numa arquitetura distribuída. Este ambiente inclui uma metodologia que separa a construção dos componentes de uma aplicação, através da programação, da construção da própria aplicação, através da configuração. A metodologia por si só não impõe restrições sobre a semântica ou a linguagem utilizada na construção dos componentes, assim como sobre o aspecto distribuído ou não da aplicação.

Os componentes do sistema são denominados módulos e são compostos por uma parte de código e dados, e um ou mais pontos de conexão externos chamados de conectores. Os módulos podem ser, por sua vez, componentes de outros módulos e não há restrição quanto a quantidade de módulos componentes de um módulo composto e, nem mesmo, quanto a quantidade de níveis de aninhamento dos módulos. A programação dos módulos não difere da programação tradicional baseada em bibliotecas externas de funções. Entretanto, a composição e interligação dos componentes da aplicação são especificadas por uma linguagem especializada de configuração^[6].

Neste artigo é apresentado o desenvolvimento do ambiente **P-RIO** com a finalidade de aproveitar as vantagens apresentadas por estes dois ambientes num só sistema. A interface oferecida pelo PVM para a programação de aplicações paralelas é implementada através de chamadas ao sistema em uma espécie de linguagem *assembly* de rede^[7], que leva à mistura da implementação com a interação dos seus componentes, deixando a maioria das decisões para o usuário^[8].

Contudo, a filosofia de criação do PVM foi a de suportar um núcleo central de primitivas acima do qual possam ser adicionadas camadas auxiliares de mais alto nível^[9]. Baseado nisto, o ambiente P-RIO estrutura aquela linguagem de rede numa interface muito mais simples, provendo uma metodologia de programação distribuída e paralela amplamente portátil. A interface apresentada ao usuário tenta ser a mais simples possível, de forma a permitir que engenheiros e cientistas, sem um grande treinamento prévio, possam construir sistemas computacionais relativamente complexos.

Na seção 2 deste artigo são discutidas as características mais relevantes apresentadas pela metodologia que implementa o ambiente P-RIO. Na seção 3 é mostrada a estrutura na qual se apoia o ambiente e o seu funcionamento dentro da máquina virtual. Na seção 4 são

apresentados os resultados de vários testes de desempenho feitos com o ambiente e sua comparação com outros sistemas. Finalmente, na seção 5 são apresentadas as conclusões e as perspectivas de trabalho futuro.

2. CARACTERÍSTICAS DO AMBIENTE P-RIO

O projeto de grandes sistemas de *software* requer mecanismos de decomposição de forma a torná-lo tratável. Ao dividir um sistema em partes menores é possível compreender mais facilmente o relacionamento entre suas diversas funcionalidades e o comportamento global. No modelo utilizado, assume-se que uma aplicação é realizada pela composição de vários módulos ou componentes, onde cada um deles requer recursos ou provê facilidades para outros módulos. Estes componentes interagem através de conectores, que representam um conjunto de linhas de comunicação, via protocolos definidos ao nível da configuração.

Este modelo permite distinguir as relações de implementação das relações de interação existentes entre os módulos. Esta é uma característica fundamental, já que quando as pessoas projetam sistemas elas tipicamente provêm uma descrição arquitetural consistindo de um conjunto de componentes computacionais e um conjunto de conexões que indicam as interações entre aqueles componentes^[10].

A arquitetura construída com a utilização de P-RIO reúne diversas características que tentam aproximar a sua utilização prática dos seus princípios conceituais, provendo, para isso, um ambiente de suporte à operação que possibilita a configuração dinâmica das aplicações além de oferecer recursos para o teste e o gerenciamento da operação. Pode-se resumir tudo isto dizendo que o ambiente P-RIO é uma metodologia de construção de *software* baseada em objetos que apresenta simplicidade, transparência e flexibilidade de uso, fatores essenciais para implementar sistemas distribuídos complexos. Ao invés de um sistema ter componentes ligados na própria descrição do código, como ocorre na programação tradicional, impõe-se uma fase de descrição da configuração, composta de informação sobre os componentes usados e suas interconexões. Isto cria uma forte distinção e isolamento entre os componentes propriamente ditos e os seus mecanismos de interligação, simplificando a especificação, teste e reutilização dos mesmos.

A METODOLOGIA

A metodologia RIO define e se baseia em dois conceitos fundamentais para a construção de um sistema: os módulos e os conectores^[1]. A estrutura básica de construção de um sistema é o módulo, o conceito de módulo se divide, por sua vez, em dois, dependendo se este é uma unidade operacional (instância) ou se é um tipo (classe) usado como fôrma para a criação das unidades operacionais. Uma classe de módulo é uma entidade que contém uma parte de código e dados e um ou mais pontos de interconexão externa. Cada instância de um módulo é uma imagem de uma classe que executa o código e possui o seu próprio estado e dados internos (Figura 1). Esta abordagem, baseada em objetos, compreende o módulo como um objeto onde uma instância do módulo é equivalente a uma instância de um objeto.



Figura 1. Um Módulo

Uma instância sofre uma ação importante para a arquitetura: a configuração. A configuração consiste na disposição e organização das instâncias e suas relações, ou seja, sua interconexão. Em um modelo tradicional de programação as ligações entre os elementos são estáticas, sendo fixadas durante a programação. Numa metodologia baseada em objetos tais ligações são potencialmente dinâmicas, realizadas em uma fase posterior à criação do sistema, ou mesmo mutáveis durante a sua operação. Esta estrutura exige um método bem definido para expressar as interações entre os módulos, que vai ser realizada através dos pontos de interconexão ou conectores.

A função dos conectores é organizar e estruturar a interface do módulo. Estes elementos concentram as atividades de comunicação e são fundamentais à configuração, pois são os pontos de "colagem" de um módulo com outro. Em outros termos, os conectores são os meios de interação entre um módulo e o seu mundo exterior. Todos os serviços usados pelo módulo ou oferecidos por ele ao sistema são acessíveis, unicamente, através dos seus conectores.

Cada conector contém um ou mais pinos de ligação. Um pino é o elemento básico por onde se realiza a comunicação, pois é a entidade através da qual os métodos ou procedimentos (programa) de um módulo irão se comunicar externamente. Pode-se visualizar os pinos como os elementos primários ativos da comunicação num módulo, enquanto que os conectores tem a função de agrupar os pinos para fins de configuração. Um conector possui um tipo ou classe que depende dos pinos nele contidos. Por sua vez, cada pino está caracterizado pelo tipo de dados e direção das mensagens, e pelo tipo de transação associada.

Desta forma, a configuração é a ligação de dois ou mais conectores, realizando, assim, a interconexão dos módulos a eles associados. Um módulo somente pode interagir diretamente com outro módulo ao qual esteja conectado. Além de expressar a interação potencial entre dois ou mais módulos, o conceito de conector pode ser usado para verificar a consistência da configuração, pois uma conexão só pode ser realizada entre conectores que contenham assinaturas de tipos compatíveis para uma interseção total ou parcial dos seus pinos.

Um conceito também associado ao de conexão é o de "grupo". Uma conexão de um conector para um grupo implica que ele está ligado a todos os outros que também pertençam ou estão conectados a esse grupo. Esta conexão especializada tem uso em diversas aplicações que requerem comunicação via disseminação de mensagens (*broadcast* ou *multicast*), como por exemplo, o suporte de servidores replicados^[11]. Os grupos podem ser explicitamente nomeados, facilitando o seu gerenciamento dinâmico.



Figura 2. Encapsulamento

Um objetivo da metodologia implementada pelo ambiente P-RIO é fornecer um ambiente não restritivo para a modelagem. Desta forma, utiliza-se um conceito genérico que permite diversas abordagens mais específicas. Este conceito genérico é a composição de módulos, que permite a criação de configurações parciais, denominadas classes compostas, a partir de classes básicas anteriormente definidas, para fins de encapsulamento e reutilização^[12] (Figura 2). Todas as classes definidas (básicas ou compostas) podem ser instanciadas e utilizadas separadamente, podendo ser executadas concorrentemente com as demais.

3. ESTRUTURA DO SISTEMA

O ambiente P-RIO está construído sobre a versão 3.3.7 do sistema PVM, garantindo-se a portabilidade do mesmo a vários sistemas operacionais e a dezenas de arquiteturas diferentes^[5]. Com isso procura-se também obter eficiência na utilização de recursos e adaptabilidade a futuras mudanças. Para efeito deste trabalho, parte-se da premissa de que não existem erros na execução normal das primitivas providas pelo sistema PVM, embora nem a verificação de resultados nem o tratamento de exceções que possam ocorrer na máquina virtual sejam deixados de lado.

O sistema PVM 3.3.7 necessita poucas modificações para oferecer o suporte requerido pelo ambiente P-RIO. Por exemplo, o conceito de “grupos” requer alterações no *Group Server* (*pvmgs*) para suportar as chamadas do Configurador P-RIO às funções *pvm_jointogroup()* e *pvm_lvfromgroup()*. As mudanças feitas são listadas em^[13] e foram propostas para inclusão no padrão PVM, devido à utilidade que as mesmas apresentam em qualquer sistema utilizando procedimentos de configuração externa.

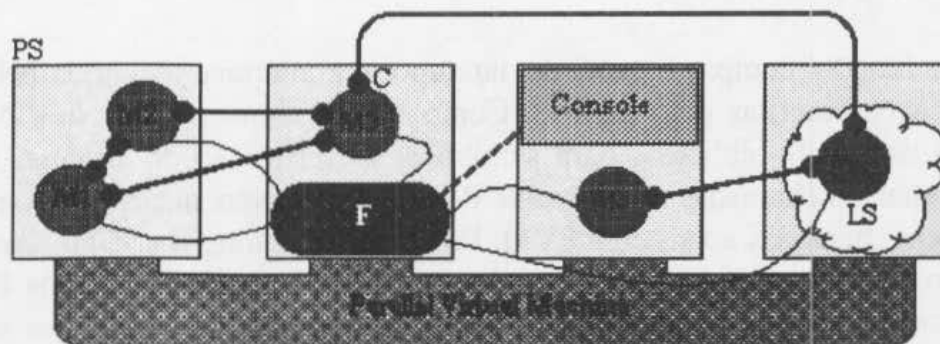


Figura 3. Estrutura do Ambiente P-RIO

O suporte de operação do ambiente P-RIO é construído sobre a plataforma computacional PVM (Figura 3). Ele consta de dois módulos principais, o Configurador (F) e a Biblioteca de Funções P-RIO, complementados por um módulo Console adicional. O módulo Console desempenha as tarefas de configuração e monitoração do ambiente criado e gerenciado pelo Configurador, enquanto que a Biblioteca é um módulo de apoio à compilação dos módulos anteriores e dos módulos M_i das aplicações dos usuários.

O CONFIGURADOR

O Configurador traduz os conceitos da metodologia P-RIO para a plataforma PVM, fornecendo serviços de configuração global e nomeação. Este módulo mantém estruturas de dados dinâmicas que são criadas e divulgadas a partir dos pedidos efetuados ou sinalizados pelo resto dos componentes. Uma vez instanciado um módulo, o Configurador toma ciência de qualquer modificação de configuração envolvendo o mesmo e atualiza suas estruturas de dados internas e externas. Desta forma, o Configurador oferece também funções de depuração: ele pode detectar a falha (ou terminação) de qualquer tarefa ou estação, mudar automaticamente as tabelas de configuração e comunicar as modificações ocorridas aos outros módulos no sistema. Além disso, ele pode gerar informes contendo a configuração existente num instante determinado.

Toda a comunicação dentro do sistema é efetuada utilizando-se as facilidades fornecidas pelo sistema PVM, dentro do objetivo de prover máxima portabilidade ao ambiente. A comunicação entre o Configurador e qualquer módulo do sistema, incluindo o Console, é em geral feita mediante o protocolo UDP: a informação trocada nestas conexões é basicamente relacionada a comandos de configuração. A comunicação entre os diversos módulos de uma aplicação através de seus conectores (C) aproveita as características do protocolo TCP, otimizando a transmissão de grandes fluxos de dados. Uma das características do ambiente RIO^[14] é que este permite a seleção do protocolo de comunicação entre dois pinos de cada conexão, sendo suportados vários protocolos padronizados e específicos. Na atual implementação do P-RIO, somente permite-se o uso do UDP ou TCP, além de disseminação não confiável. Em máquinas usando estruturas especializadas de interconexão, protocolos particulares serão transparentemente suportados.

A BIBLIOTECA DE FUNÇÕES

A biblioteca de funções cumpre o papel de dar suporte à interface requerida pela metodologia RIO e às aplicações escritas pelo usuário. Componentes como o Console e o Configurador também fazem uso desta biblioteca para simplificar a construção do sistema. As funções da biblioteca mapeiam as chamadas do ambiente P-RIO, disponíveis nos níveis de programação e configuração, em chamadas ao sistema PVM. Procurou-se manter o código simples e eficiente de modo a não afetar o desempenho originalmente oferecido pela plataforma PVM. Entre as funções implementadas por esta biblioteca pode-se citar aquelas referentes à configuração dinâmica do sistema (estações, módulos, conexões, etc.), envio e recepção de mensagens (primitivas síncronas, assíncronas, *selects*, *timeouts*, etc.), administração de grupos, depuração, e outras^[15]. As funções da biblioteca são descritas detalhadamente em^[13].

Dentro do conjunto de primitivas não existem chamadas restritivas, todas as funções da biblioteca são utilizáveis diretamente por qualquer módulo de uma aplicação. Além disso, como um auxílio à depuração^[16], foi estabelecido que um erro ocorrido em qualquer destas chamadas gera uma mensagem auto-explicativa baseada no valor da variável global "errno" ou, em casos mais críticos, lista o estado atual da instância de módulo aonde o problema foi detectado.

O MÓDULO CONSOLE

O módulo console é a interface com o usuário que permite a configuração, monitoração e depuração das aplicações no ambiente. Além de iniciar a execução do Configurador como um processo PVM puro, este módulo traduz cada linha de comandos, expressos em uma linguagem intermediária de configuração^[17], em chamadas ao sistema P-RIO. Uma vez configurada e em execução a aplicação, o Console cuida da captura das mensagens provenientes do sistema, podendo também executar atividades de reconfiguração dinâmica.

A serialização das saídas *printf-like* através dos *daemons* PVM é um gargalo visível que o console tenta solucionar mediante um *polling* contínuo do sistema. Os comandos de configuração e depuração existentes na linguagem intermediária de configuração (aceitos pelo console) são descritos em ^[13].

AS APLICAÇÕES

Módulos a serem executados no ambiente P-RIO devem incluir no seu código C a declaração `#include <prio.h>` e ser ligados com a biblioteca *libprio.a*. Além disso, uma chamada à uma função especial, `MD_HEADER()`, deve ser feita no início da execução do código de modo a inserir a instância no ambiente. Não existem restrições com respeito à ordem de criação das instâncias da aplicação, sendo contudo aconselhável a iniciação dos pinos de comunicação, através de chamadas especiais, antes do início da fase de processamento. Por medida de conveniência, pode-se estabelecer um ponto de sincronização para o início do processamento de todas as instâncias de uma aplicação usando-se a primitiva `SYNC_MD()` ou `SYNC_BARRIER()`.

O ambiente P-RIO suporta duas primitivas básicas de comunicação: síncrona (tipo RPC) e assíncrona, e funções auxiliares de temporização (*timeout*) e propagação de exceções. Além disso, são disponíveis, como em outras metodologias de programação concorrente contemporâneas, construções do tipo *select* com guardas. Estas facilidades facilitam a programação de mecanismos e funções típicas, tais como distribuição de carga, tolerância a falhas, paralelização dinâmica, etc.

A concepção do suporte de operação do ambiente P-RIO permite que todas as ferramentas de depuração originais do PVM (XPVM, HeNCE, Xab, DoPVM, etc.) funcionem como ferramentas auxiliares para este ambiente. Desta forma, otimizações específicas à uma aplicação podem ser realizadas através de chamadas a ferramentas e serviços PVM, especializando a programação dos módulos, de forma a obter componentes mais eficientes.

Finalmente, deve-se mencionar que a instalação do P-RIO não difere essencialmente da de qualquer outra aplicação PVM, sendo imediata em qualquer plataforma PVM 3.3.x^[13].

4. AVALIAÇÃO

Com o fim de ter uma medida quantitativa do desempenho alcançado pelo sistema, tanto em tarefas de comunicação, como em tarefas de processamento paralelo e distribuído, são apresentados, a seguir, os resultados de duas experiências. A primeira delas obtém o tempo empregado em uma comunicação *round-trip* entre dois módulos, instanciados em diferentes estações SUN4, e faz a comparação desses valores com os obtidos em outros sistemas sob condições similares ou mesmo superiores. A Tabela 1 resume os valores médios obtidos. Os resultados mostrados são compatíveis com os apresentados em outros artigos^{[18], [19]}.

TEMPO de COM.	0 bytes	1 bytes	100 bytes	1 Kbyte
Sockets	2.8	2.9	3.3	5.9
RPC	-	5.7	-	-
PVM	6.8	7.1	8.0	10.4
RIO	-	9.9	-	-
P-RIO	5.7	6.3	7.5	10.5

Tabela 1. Tempos de Round-Trip (ms.)

Pode-se perceber que, mesmo sendo o ambiente P-RIO baseado nas primitivas fornecidas pelo sistema PVM, os tempos empregados na comunicação *round-trip* são menores para mensagens inferiores a 1 Kb e bastante similares para mensagens mais longas. Isto se deve, principalmente, ao menor número de chaveamentos para as tarefas P-RIO, justificado pela maior utilização de CPU durante o envio ou recepção de mensagens.

TEMPO DE EXEC.	Cálculo de π
1 Estação	74.6
2 Estações	37.5
3 Estações	26.1
4 Estações	20.0
5 Estações	16.9
6 Estações	14.8
7 Estações	12.9
8 Estações	11.4
9 Estações	10.2
10 Estações	9.1

Tabela 2. Tempos de Execução (seg.)

A segunda experiência estabelece o tempo para o cálculo do valor de π , com 15 casas decimais, através da integração numérica da equação $x^2 + y^2 = 1$ entre $0 \leq x \leq 1$. A Tabela 2 mostra os tempos médios quando se utiliza de 1 a 10 estações SUN4. Uma descrição mais completa e a listagem dos módulos e dos arquivos de configuração são apresentadas em ^[20].

Deve-se ressaltar que qualquer aplicação que utilize uma rede distribuída de computadores deve ter granularidade relativamente grossa. Em outras palavras, é desejável que a sua razão computação/comunicação seja a mais alta possível. Devido a isto, o tempo de comunicação deve desempenhar um papel mínimo no rendimento global da aplicação, como no exemplo apresentado. Pelos resultados obtidos pode-se dizer que o ambiente PVM é mais adequado àquelas aplicações distribuídas que não necessitam de um elevado desempenho^[21], especialmente quanto à velocidade de transferencia de dados. Isto, contudo, é bastante aceitável na maioria das situações.

No exemplo a seguir, pode-se observar claramente que o ambiente P-RIO facilita a configuração de aplicações paralelas, além de prover um ambiente de programação distribuída simples e flexível. As aplicações que podem ser modeladas são inúmeras, devendo somente ser resolvidos os problemas de adequação que possam ocorrer entre a razão computação/comunicação da aplicação com a velocidade fornecida pela rede de suporte.

EQUAÇÃO DE POISSON

É apresentada a título de exemplo, uma resolução da Equação de Poisson, tentando utilizar a maioria dos conceitos fornecidos pelo ambiente P-RIO (*selects*, grupos, sincronização, etc.). A Equação de Poisson é uma simples e importante equação da Física, expressa como:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y) \quad [1]$$

onde o problema consiste em encontrar a função $u(x, y)$ que satisfaça a equação [1] dentro de uma região fechada D e condições dadas sobre os limites de D . O algoritmo^[22] para efetuar esta tarefa é descrito a seguir:

- Os valores da função contínua $f(x, y)$ são armazenados numa matriz discreta $M \times M$;
- A cada um dos elementos da matriz anterior, é aplicada a aproximação discreta de [1]

$$x_{i,j} = 0.25(x_{i+1,j} + x_{i-1,j} + x_{i,j+1} + x_{i,j-1} - f(i, j) / h^2) \quad [2]$$

onde i e j são os índices da matriz e h o nível de discretização;

- O passo (b) é aplicado repetidas vezes até que $\Delta x_{i,j}$ ($x_{i,j} \text{ NEW} - x_{i,j} \text{ OLD}$) seja menor que um valor ϵ para todos os pontos da matriz.

No exemplo, a solução é obtida em forma paralela mediante a distribuição da matriz original sobre um número determinado de processadores. Mesmo sendo este um algoritmo simples, a

sua paralelização não é trivial. Ao dividir a matriz original em sub-matrizes e associar cada uma delas a um processador diferente, um problema surge: o grande compartilhamento de dados entre as sub-matrizes vizinhas. Isto gera uma granularidade muito fina, que contradiz o anteriormente dito sobre a tendência das aplicações PVM em um sistema distribuído.

O sistema implementado tenta dar transparência ao processo de paralelização e, mais do que preocupar-se com a fidelidade dos resultados, procura demonstrar a utilização do maior número possível de primitivas P-RIO. O sistema é composto por três módulos:

1. O módulo "application", encarregado de criar a matriz inicial, chamar o procedimento de cálculo e receber os resultados uma vez terminado o processo de computação;
2. O módulo "parallel", que recebe a chamada remota tendo como dados a matriz original. Este módulo particiona a matriz e entrega cada porção a um processador diferente, recebendo posteriormente os resultados do processo de cálculo;
3. O módulo "worker", que executa o cálculo. Ele é replicado em 4 instâncias, que recebem uma parte da matriz original e a transformam mediante a utilização da equação [2] (realizando, também, as transferências de dados entre os módulos da mesma classe). Ao final, eles retornam os resultados de forma ordenada ao módulo "parallel" e ao módulo "application", usando para isto os conceitos de grupo e *select*.

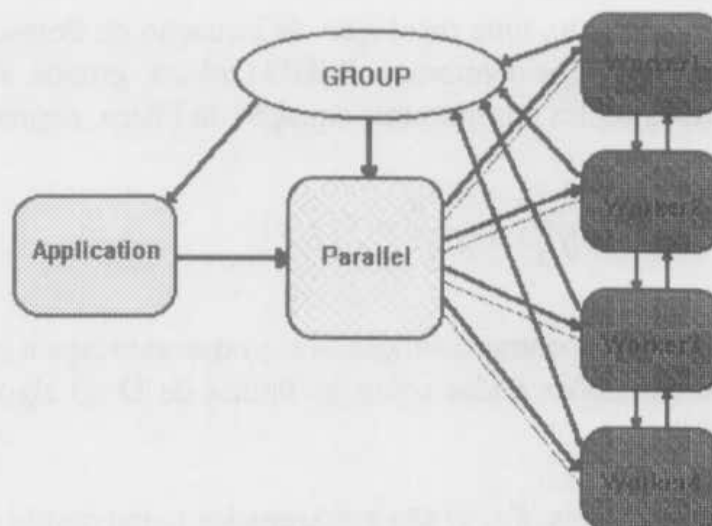


Figura 4. Resolução da Equação de Poisson

A listagem com a programação dos módulos é fornecida em ^[20]. O arquivo de configuração para esta aplicação, escrito na respectiva linguagem de configuração^[6], é mostrado a seguir:

```
class Poisson_Equation {
  station PE[6] = {
    "Hydra", "Cygnus", "Gemini", "Orion", "Tucana", "Draco";
  connector Matrix_Type {
    pin Matrix out pin_output;
  };
  connector Data_Type {
    pin Data out pin_output;
  };
};
```

```

connector Comm_Type {
    pin Data out pin_output1;
    pin Cont out pin_output2;
};
connector Link_Type {
    pin Link out pin_output;
    pin Link in pin_input;
};
group BackData_Group;
class Application_Modules {
    class Application {
        connector      Matrix_Type channel_out;
        pair connector Data_Type  channel_in;
        extern code C  "application";
    } module1;
    class Parallel {
        connector      Comm_Type  fanout[4];
        pair connector Matrix_Type channel_in;
        pair connector Data_Type  channel_grp;
        extern code C  "parallel";
    } module2;
    class Workers {
        pair connector Comm_Type channel_in;
        connector      Link_Type upper;
        pair connector Link_Type lower;
        connector      Data_Type channel_out;
        extern code C  "worker";
    } moduleN[4];
    link module1.channel_out module2.channel_in;
    for i = 0 to 3 {
        link module2.fanout[i] moduleN[i].channel_in;
        link moduleN[i].channel_out BackData_Group;
    }
    for i = 0 to 2
        link moduleN[i].upper moduleN[i+1].lower;
    link BackData_Group module1.channel_in;
    link BackData_Group module2.channel_grp;
    instantiate module1 at EP[4];
    instantiate module2 at EP[5];
    for i = 0 to 3
        instantiate moduleN[i] at EP[i];
} system;
instantiate system;
};

```

5. CONCLUSÃO

Neste artigo foram apresentados alguns aspectos do ambiente **P-RIO**. Este ambiente é centrado em uma metodologia de *software* que privilegia a construção de sistemas concorrentes através da composição de módulos. O suporte desta metodologia sobre a plataforma **PVM**, disponível em um grande número de arquiteturas de *hardware*, foi também apresentado. A estratégia desta fusão visa aliar as abstrações e mecanismos de alto nível oferecidos pela metodologia associada ao ambiente RIO, às características de portabilidade entre arquiteturas de *hardware* propiciadas pela máquina virtual PVM, que oferece mecanismos elementares de programação. Desta forma, pretende-se tornar a programação paralela e distribuída mais acessível a programadores não especialistas e, também, oferecer conceitos e mecanismos de construção de *software* baseados no paradigma de configuração,

que incentivem a modularidade e re-usabilidade do *software*. Foram discutidas as características mais relevantes apresentadas por esta metodologia, o suporte de operação a ela associado, e a sua construção e funcionamento a partir da máquina virtual. Foram também apresentados os resultados de vários testes de desempenho que verificam e confirmam os objetivos propostos, além de exemplos de aplicações que podem se beneficiar deste ambiente. Pode-se concluir que o ambiente P-RIO facilita a construção e o suporte de aplicações paralelas e distribuídas, além de prover uma metodologia de programação concorrente simples e flexível.

Uma das características que o ambiente P-RIO ainda não aproveita totalmente, e que também não é fornecida pelo sistema PVM, é a sua potencialidade para a construção de aplicações tolerantes a falhas. A construção de tais aplicações é uma tarefa complexa, mas a metodologia embutida no ambiente facilita o seu suporte. Fica então aberta a possibilidade de se incrementar a biblioteca P-RIO para que forneça as primitivas básicas para a configuração automática de aplicações tolerantes a falhas. Isto permitirá o aproveitamento das mesmas na replicação dos componentes chave do sistema, como por exemplo, o Configurador.

O balanceamento de carga estático é o único atualmente disponível no sistema PVM. Contudo, na construção de aplicações paralelas é também desejável ter a possibilidade de balanceamento automático de carga. Assim, não está fechada a possibilidade de se construir funções de biblioteca que permitam o balanceamento de carga dinâmico na máquina virtual.

6. BIBLIOGRAFIA

- [1] Werner J., Loques O., *Ambiente RIO: Metodologia e Suporte para Sistemas Configuráveis*, XX SEMISH da Reunião Anual da Sociedade Brasileira de Computação, Setembro 1993.
- [2] Geist A., Beguelin A., Dongarra J., Jiang W., Manchek R., Sunderam V., *PVM: Parallel Virtual Machine - A Users' Guide and Tutorial for Networked Parallel Computing*, The MIT Press - Cambridge, Agosto 1994.
- [3] Beguelin A., Dongarra J., Geist A., Jiang W., Manchek R., Moore K., Sunderam V., *The PVM Project*, ORNL/TM-12187, Agosto 1993.
- [4] Dongarra J., Geist A., Manchek R., Sunderam V., *Integrated PVM Framework Supports Heterogeneous Network Computing*, Computers in Physics, Janeiro 1993.
- [5] Geist A., Beguelin A., Dongarra J., Jiang W., Manchek R., Sunderam V., *PVM 3 Users's Guide and Reference Manual*, ORNL/TM-12187, Setembro 1994.
- [6] Malucelli V., *Ambiente Rio - Linguagem de Configuração*, Relatório Técnico DEE-PUC/RJ, Maio 1994.
- [7] Manchek R., *An Introduction to PVM (Parallel Virtual Machine)*, University of Tennessee, Agosto 1993.

- [8] Sunderam V., *PVM: A Framework for Paralell Distributed Computing*, Concurrency: Practice & Experience, Dezembro 1990.
- [9] Geist A., *The PVM Concurrent Computing System: Evolution, Experiences, and Trends*, J. Parallel Computing, Abril 1993.
- [10] Allen R., Garlan D., *Beyond Definition/Use: Architectural Interconnection*, Workshop on Interface Definition Languages, Portland-EUA, Janeiro 1994.
- [11] Brito O., Malucelli V., Loques O., *Tolerância a Falhas no Ambiente RIO*, V Simposio Brasileiro de Computadores Tolerantes a Falhas, Outubro 1993.
- [12] Ning J., Miriyala K., Kozaczynski W., *An Architecture-driven, Business-specific, and Component-based Approach to Software Engineering*, III International Conference of Software Reuse - Brasil, 1994.
- [13] Carrera E., Loques O., Leite J., *Implementação da Metodologia RIO sobre PVM*, Relatório Técnico DEE-PUC/RJ, Novembro 1994.
- [14] Sztajnberg A., Loques O., *O Sistema de Comunicação Multi-Protocolo do Ambiente RIO*, V Simposio Brasileiro de Computadores Tolerantes a Falhas, Outubro 1993.
- [15] Werner J., *Um Ambiente para a Construção, Configuração e Operação de Sistemas Distribuídos*, Dissertação de Mestrado DEE-PUC/RJ, Janeiro 1995.
- [16] Hong W., Black J., Manning E., *A Framework for Distributed Debugging*, IEEE Software p106-115, Janeiro 1990.
- [17] Sztajnberg A., Malucelli V., *Manual do Usuário do Ambiente RIO*, Relatório Técnico DEE-PUC/RJ, Abril 1994.
- [18] Sztajnberg A., Loques O., Leite J., *Implementação e Testes do Sistema de Comunicação do Ambiente RIO*, XII Simposio Brasileiro de Redes de Computadores, Maio 1994.
- [19] Douglas C., Mattson T., Schultz M., *Parallel Programming Systems for Workstation Clusters*, YALEU/DCS/TR-975, Agosto 1993.
- [20] Carrera E., Loques O., Leite J., *Aplicações Paralelas no Ambiente P-RIO*, Relatório Técnico DEE-PUC/RJ, Novembro 1994.
- [21] Grant B., Skjellum A., *The PVM Systems: An In-Depth Analysis and Documenting Study - Concise Edition*, Lawrence Livermore National Laboratory, Setembro 1992.
- [22] Hwang K., Briggs F., *Arquitetura de Computadoras y Procesamiento Paralelo*, McGraw-Hill México, 1988.